

① FCFS

⇒ #include <stdio.h>

void swap (int *xp, int *yp) {

int temp = *xp;

*xp = *yp;

*yp = temp;

}

void bubbleSort (int arr[], int n) {

int i, j;

int swapped;

for (i = 0; i < n-1; i++) {

swapped = 0;

for (j = 0; j < n-i-1; j++) {

if (arr[j] > arr[j+1]) {

swap (&arr[j], &arr[j+1]);

swapped = 1;

}

}

if (swapped == 0) break;

}

}

void main() {

{

int n, ct[n], tat[n], current_time = 0;

int wt[n], sumwt = 0, sumtat = 0;

printf("enter the no of processes");

scanf("%d", &n);

int at[n], bt[n], pid[n];


```
printf("Enter the at & bt");
for(int i=0; i<n; i++){
    scanf("%d", &at[i]);
    scanf("%d", &bt[i]);
    pid[i] = i+1;
}

bubbleSort(at, n);
for(int i=0; i<n; i++){
    if(current_time < at[i]){
        current_time = at[i];
    }
    ct[i] = current_time + bt[i];
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    current_time = ct[i];
    sumwt += wt[i];
    sumtat += tat[i];
}

int awt = sumwt / n;
int atat = sumtat / n;
printf("These are the corresponding times:");
for(int i=0; i<n; i++){
```

```
printf("pid: %d in at: %d in bt: %d in\n",
    ct: %d in wt: %d in tat: %d\n",
    awt: %d in atat: %d\n",
    pid[i], at[i], bt[i], ct[i], wt[i],
    tat[i], awt, atat);
}
```

output:

enter the number of processes 2
enter the at & bt 10 20 10 20
these are the pid and corresponding time

pid: 1
at: 10
bt: 20
ct: 30
wt: 0
tat: 20
awt: 10
atat: 30

pid: 2
at: 10
bt: 20
ct: 50
wt: 20
tat: 40
awt: 10
atat: 30

output:

enter the number of processes 2
enter the at & bt 0 10 10 20
these are the pid and corresponding times:

pid: 1	pid: 2
at: 0	at: 10
bt: 10	bt: 20
ct: 10	ct: 30
wt: 0	wt: 0
tat: 10	tat: 20
awt: 0	awt: 0
atat: 15	atat: 15

② SJP

```
> #include <stdio.h>
```

```
int main() {
```

```
    int n, i, j;
```

```
    int at[20], bt[20], tat[20], wt[20];
```

```
    int comp[20] = {0};
```

```
    int current_time = 0, completed_count = 0;
```

```
    int min_bt, index;
```

```
    float avg_wt = 0;
```

```
    float avg_tat = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for(i=0; i<n; i++) {
```

```
        printf("Enter AT and BT for p%d: ",
```

```
            i+1);
```

```
        scanf("%d %d", &at[i], &bt[i]);
```

```
    }
```

```
    while(comp < n) {
```

```
        min_bt = 9999;
```

```
        index = -1;
```

```
        for(i=0; i<n; i++) {
```

```
            if (at[i] <= current_time &&
```

```
                comp[i] == 0) {
```

```
                if (bt[i] < min_bt) {
```

```
                    min_bt = bt[i];
```

```
                    index = i;
```

```
            }
```

```
        }
```

```
    }
```

```
    if (index == -1)
```

```
        current_time ++;
```



```

else {
    ct[index] = current_hnu + bt[index];
    tat[index] = ct[index] - at[index];
    wt[index] = tat[index] - bt[index];
    current_hnu = ct[index];
    comp[index] = 1;
    completed = count++;
    avg_wt += wt[index];
    avg_tat += tat[index];
}
}
avg_wt /= n;
avg_tat /= n;
printf("PID AT BT CT TAT WT\n");
for(i=0; i<n; i++)
    printf("P%d AT%d BT%d CT%d TAT%d WT%d\n", i, at[i], bt[i], ct[i], tat[i], wt[i]);
printf("AV WT = %.2f, avg wt);
printf("AV tat = %.2f, avg tat);

return 0;
}

```

output:

case 1:

Enter number of processes = 2
 Enter at and bt for P1: 0 5
 Enter at and bt for P2: 1 4

PID	AT	BT	CT	TAT	WT
P1	0	5	3	5	0
P2	1	4	9	8	4

AV WT = 2.00

AV TAT = 6.5

case 2:

enter number of processes = 2
 enter at and bt for P1: 0 3
 enter at and bt for P2: 0 2

PID	AT	BT	CT	TAT	WT
P1	0	3	5	5	2
P2	0	2	2	2	0

AV WT = 1.00

AV TAT = 3.50

Run 13/12/20

③ Priority Scheduling

⇒ #include <stdio.h>

```
void nonPru(int n, int* pid, int* at, int* bt, int*
    priority, int* ct, int* tat, int* wt,
    int* completed) {
```

```
    int time = 0, count = 0;
```

```
    while (count < n) {
```

```
        int idx = -1;
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (at[i] <= time || completed[i]) {
```

```
                if (idx == -1 || priority[i] < priority
                    [idx])
```

```
                    idx = i;
```

```
            }
```

```
        }
```

```
        if (idx != -1) {
```

```
            time += bt[idx];
```

```
            ct[idx] = time;
```

```
            tat[idx] = ct[idx] - at [idx];
```

```
            wt[idx] = tat[idx] - bt[idx];
```

```
            completed[idx] = 1;
```

```
            count++;
```

```
        }
```

```
    } else time++;
```

```
    }
```

```
}
```

```
void pru(int n, int* pid, int* at, int* bt, int*
    pri, int* ct, int* tat, int* wt, int* comp) {
```

```
    int time = 0, count = 0;
```

```
    int rt[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        rt[i] = bt[i];
```

```
        comp[i] = 0;
```

```
    }
```

```
    while (count < n) {
```

```
        int idx = -1;
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (at[i] <= time || compcompleted[i] == 0) {
```

```
                if (idx == -1 || pri[i] < pri[idx]) idx = i;
```

```
            }
```

```
        }
```

```
        if (idx == -1) time++;
```

```
        else {
```

```
            rt[idx] = -1;
```

```
            time++;
```

```
            if (rt[idx] != 0) {
```

```
                ct[idx] = time;
```

```
                tat[idx] = ct[idx] - at[idx];
```

```
                wt[idx] = tat[idx] - bt[idx];
```

```
                comp[idx] = 1;
```

```
                count++;
```

```
            }
```

```
        }
```

```
}
```


Date: / /
Page:

```

int main() {
    int n;
    printf("Enter no of processes:");
    scanf("%d", &n);
    int at[n], bt[n], pri[n], pid[n], ct[n], tat[n], wt[n], comp[n];
    int ch;
    printf("1. Non-preemptive 2. preemptive");
    scanf("%d", &ch);
    for(int i=0; i<n; i++) {
        printf("AT, BT, priority for process %d: ", i+1);
        scanf("%d %d %d", &at[i], &bt[i], &pri[i]);
        comp[i] = 0; pid[i] = i+1;
    }
    switch(ch) {
        case 1: nonpre(n, pid, at, bt, pri, ct, tat, wt, comp);
                break;
        case 2: pre(n, pid, at, bt, pri, ct, tat, wt, comp);
                break;
        default: printf("Invalid");
    }
    return 1;
}

float atot = 0, awt = 0;
for(i=0; i<n; i++) {
    atot += tat[i];
    awt += wt[i];
}

atot /= n;
awt /= n;

```

Date: / /
Page:

```

printf("m pid at bt pri ct tat wt");
for(i=0; i<n; i++) {
    printf(" %d %d %d %d %d %d %d %d",
           pid[i], at[i], bt[i], pri[i],
           ct[i], tat[i], wt[i]);
}

printf(" atot: %f", atot);
printf(" awt: %f", awt);
return 0;
}

```

Output: Non-preemptive

Enter no of processes: 3

1. Non-preemptive 2. preemptive 1

AT, BT, priority for process 1: 1 3 3

AT, BT, priority for process 2: 1 5 1

AT, BT, priority for process 3: 3 3 2

pid	at	bt	pri	ct	tat	wt
1	0	3	3	3	3	0
2	1	5	1	8	7	2
3	3	3	2	11	8	5

atot: 6.000000

awt: 2.333333

preemptive

Enter no of processes: 3

1. Non-preemptive 2. preemptive 2

AT, BT, priority for process 1: 0 3 3

AT, BT, priority for process 2: 1 5 1

AT, BT, priority for process 3: 3 3 2

pid at bt pri at tat wt

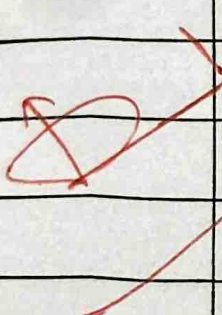
1 20 3 3 47 11 3 8

2 1 5 1 6 3 6

3 3 3 2 9 6 3

atat: 7.333333

awt: 3.666666



g

(4)

Round-Robin Scheduling

→ #include <stdio.h>

```
void robin (int n, int *at, int *bt, int *rt, int *ct,
            int tq) {
```

```
    int time = 0, comp = 0;
```

```
    int vis[n];
```

```
    for (int i = 0; i < n; i++) vis[i] = 0;
```

```
    while (comp < n) {
```

```
        int found = 0;
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (at[i] <= time && rt[i] > 0) {
```

```
                found = 1;
```

```
                if (rt[i] > tq) {
```

```
                    time += tq;
```

```
                    rt[i] -= tq;
```

```
                } else {
```

```
                    time += rt[i];
```

```
                    rt[i] = 0;
```

```
                    ct[i] = time;
```

```
                    comp++;
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
    if (found == 0) time++;
```

```
}
```

```
}
```



```

int main()
{
    int n, tq;
    printf("Enter no of processes\n");
    scanf("%d", &n);
    int pid[n], at[n], bt[n], rt[n], ct[n];
    printf("Enter quantum time:\n");
    scanf("%d", &tq);
    for (int i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("in process %d in", pid[i]);
        printf("AT:");
        scanf("%d", &at[i]);
        printf("BT:");
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }
    round_robin(n, at, bt, rt, ct, tq);
    printf("in PID it AT it BT it comp RT in");
    for (int i = 0; i < n; i++) {
        printf("%d it %d it %d it %d in", pid[i],
            at[i], bt[i], ct[i]);
    }
    return 0;
}

```

Output :

Enter no of processes: 3

enter quantum finu: 2

рзюсм 1

$$A^T : 0$$
$$BT = 20$$

process 2

$$A \hat{T} \leq 2$$

BT : 5

प्रवाल, ३

$$AT : 8$$

BT: 6

P10	at	bt	ct	fast	wt
1	0	20	31	31	11
2	2	5	13	11	6
3	8	6	19	11	5

$$atat = 17.666666$$

aulf: 7-333333

[illegible]
$$t_0 = 6.$$

For 27/3/26

⑤ Multilevel queue scheduling

```
#include <stdio.h>
#include <string.h>
#define max 100

typedef struct {
    int pid;
    int arrival, burst, remainings, completion,
    waiting, turnaround;
} process;

typedef queue {
    int items[max];
    int front, rear;
} Queue;

void initQueue (Queue *q) {
    q->front = q->rear = -1;
}

int isEmpty (Queue *q) {
    return (q->front == -1);
}

void enqueue (Queue *q) {
    if (q->rear == max-1) return;
    if (q->front == -1) q->front = 0;
    q->items[q->rear] = 1;
    q->rear++;
}

void dequeue (Queue *q) {
    if (isEmpty(q)) return -1;
    int val = q->items[q->front];
    if (q->front == q->rear)
        q->front = q->rear = -1;
}
```

```
else q->front++;
return val;
}

void sortByArrival (process p[], int n) {
    process temp;
    for (int i=0; i<n-1; i++) {
        for (int j=i+1; j<n; j++) {
            if (p[i].arrival > p[j].arrival) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}

void main() {
    int n;
    process p[max];
    printf("Enter the number of processes");
    scanf("%d", &n);
    for (int i=0; i<n; i++) {
        printf("Enter process %d m", i+1);
        printf("Arrival time");
        scanf("%d", &p[i].arrival);
        printf("Burst time");
        scanf("%d", &p[i].burst);
        printf("Type (system/user):");
        scanf("%s", p[i].type);
        p[i].pid = i+1;
        p[i].remainings = p[i].burst;
    }
}
```



```

sort by arrival (P, n);
queue system, user;
init_queue (&system);
init_queue (&user);
int time = 0, computed = 0, i = 0;
int current = -1;
int gantt[max], g_time[max];
int g_index = 0;
while (computed < n) {
    while (i < n && P[i].arrival <= time) {
        if (strcmp(P[i].type, "system")
            == 0)
            enqueue (&system, i);
        else
            enqueue (&user, i);
        i++;
    }
    if (current == -1) {
        if (!isempty (&system))
            current = dequeue (&system);
        else if (!isempty (&user))
            current = dequeue (&user);
        else {
            time++;
            continue;
        }
    }
    gantt[g_index] = P[current].pid;
    g_time[g_index] = time;
    g_index++;
    P[current].remaining--;
    time++;
}

```

```

if (P[current].remaining == 0) {
    P[current].completion = time;
    computed++;
    current = -1;
}
}
g_time[g_index] = time;
float twt = 0, ttat = 0;
for (int i = 0; i < n; i++) {
    P[i].turnaround = P[i].completion -
        P[i].arrival;
    P[i].waiting = P[i].turnaround - P[i].burst;
    twt += P[i].waiting;
    ttat += P[i].turnaround;
}
printf ("\n Gantt chart: \n");
for (int i = 0; i < g_index; i++) {
    printf ("%d |", gantt[i]);
}
printf ("\n %d %g time [0];",
    for (int i = 0; i <= g_index; i++) {
        printf ("%d", g_time[i]);
    }
printf ("\n PID | type | AT | BT | CT | TAT | W T | \n");
for (int i = 0; i < n; i++) {
    printf ("%d | %d | %d | %d | %d | %d | %d | %d | \n",
        P[i].pid, P[i].type, P[i].arrival, P[i].burst,
        P[i].completion, P[i].turnaround,
        P[i].waiting);
}
printf ("TWT: %.2f", twt/n);
printf ("TAT: %.2f", ttat/n);
}

```


Output:

Enter no of processes :- 3

Process: 1

Arrival time: 0

Burst time: 5

type: system

Process: 2

Arrival time: 2

Burst time: 6

type: ~~user~~ system

Process 3

Arrival time: 3

Burst time: 8

type: user

PID	type	AT	BT	CT	TAT	WT
P1	system	0	5	9	9	9
P2	system	2	6	11	9	3
P3	user	3	8	19	16	8

AWT: 3.67

ATAT: 10.0

Process	system	user	Process
0-2	<P2, P1>	(1)	P1
2-4	<P1, P2>	(P3)	P2
4-6	(P2, P1)	(P3)	P1
6-8	(P1, P2)	(P3)	P2
8-9	(1)	(P3)	P1
9-11	(1)	(P3)	P2
11-19	(1)	(P3)	P3

⑥ RMS

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

struct process {
    int pid, burst, period, remaining;
};

int gcd (int a, int b) {
    if (b == 0) return a;
    return gcd (b, a % b);
}

int lcm (int a, int b) {
    return (a * b) / gcd (a, b);
}

int main() {
    int n;
    printf("Enter the no of processes");
    scanf("%d", &n);
    struct process p[n];
    printf("Enter the CPU burst times: \n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &p[i].burst);
        p[i].pid = i + 1;
        p[i].remaining = 0;
    }
    printf("LCM = %d \n", hyperPeriod);
    printf("RMS: \n");
    printf("PID | burst | period | \n");
    for (int i = 0; i < n; i++) {
        printf("%d | %d | %d |", p[i].pid, p[i].burst, p[i].period);
    }
}
```

```
double u = 0.0;
for (int i = 0; i < n; i++) {
    u += (double) p[i].burst / p[i].period;
}

double bound = n * (pow(2.0, 1.0 / n) - 1);
printf("n %.6f <= %.6f => %.8f \n", u, bound, (u <= bound ? "true" : "false"));
printf("scheduling occurs for %d ms \n", hyperPeriod);
int prev_chosen = -2;
for (int t = 0; t < hyperPeriod; t++) {
    for (int i = 0; i < n; i++) {
        if (t % p[i].period == 0) {
            p[i].remaining = p[i].burst;
        }
    }

    int chosen = -1;
    int min_period = 99999;
    for (int i = 0; i < n; i++) {
        if (p[i].remaining > 0 && p[i].period < min_period) {
            min_period = p[i].period;
            chosen = i;
        }
    }

    if (chosen != prev_chosen) {
        if (chosen != -1) {
            printf("%d ms onwards : process %d running \n", t, p[chosen].pid);
        }
    }
    else {
        printf("%d ms onwards : CPU is idle \n", t);
    }
}
```



```

prev_chosen = chosen;
{
    if (chosen != -1) {
        p[chosen].remaining--;
    }
}

```

```

return 0;
}

```

output:

Enter the no of processes: 3

Enter the CPU burst time:

1 2 1

Enter the time periods:

4 5 10

LCM = 20

RMS:

PID	Burst	Period
1	1	4
2	2	5
3	1	10

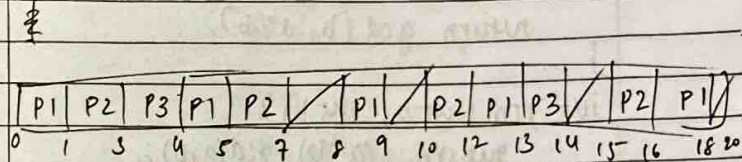
$0.75 \leq 0.779763 \Rightarrow$ true

scheduling for 20 ms

0 ms : P1
 1 ms : P2
 3 ms : ~~idle~~ P3
 4 ms : P1
 5 ms : P2
 7 ms : idle
 8 ms : P1
 9 ms : idle

10 ms : P2
 12 ms : P1
 13 ms : P3
 14 ms : ~~idle~~
 15 ms : ~~idle~~ P2
 16 ms : ~~P2~~ P1
 18 ms : idle

gantt chart:



④ EDF

```
#include <stdio.h>
struct Task {
    int pid, burst, deadline, period, sum, abs_dl;
};
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}
int main() {
    int n;
    printf("Enter no of processes: ");
    scanf("%d", &n);
    struct Task tks[n];
    int hyperperiod = 1;
    for (int i = 0; i < n; i++) {
        printf("Enter PID, Burst, Deadline, and Period for process %d: ", i + 1);
        scanf("%d %d %d %d", &tks[i].pid, &tks[i].burst, &tks[i].deadline, &tks[i].period);
        tks[i].sum = 0;
        tks[i].abs_dl = 0;
        hyperperiod = lcm(hyperperiod, tks[i].period);
    }
    printf("In PID | Burst | Deadline | Period |");
```

```
for (int i = 0; i < n; i++) {
    printf("%d | %d | %d | %d |",
           tks[i].pid, tks[i].burst, tks[i].deadline,
           tks[i].period);
}
printf("\n scheduling occurs for %d ms in %d",
       n, hyperperiod);
int prev_selected = -2;
for (int i = 0; i < n; i++) {
    if (time % tks[i].period == 0) {
        if (tks[i].sum > 0) {
            printf("%d ms : Deadline missed by process %d in %d, time, tks[i].pid);
        }
        tks[i].sum += tks[i].burst;
        tks[i].abs_dl = time + tks[i].deadline;
    }
}
int selected = -1;
int min_dk = 99999;
for (int i = 0; i < n; i++) {
    if (tks[i].sum > 0 && tks[i].abs_dl < min_dk) {
        min_dk = tks[i].abs_dl;
        selected = i;
    }
}
if (selected != prev_selected) {
    if (selected != -1) {
        printf("%d ms onwards : process %d running in %d, time, tks[selected].pid);
    }
}
```


die?

```
printf("%d ms onwards : CPU is  
idle\n", time);
```

```
}
```

```
proc_selected = selected;
```

```
}
```

```
if (select != -1) HasSelected = true;
```

```
}
```

```
return 0;
```

```
}
```

output:

Enter no of process : 3

Enter PID, Burst, Deadline, and Period for process 1:

1 1 3 6

process 2:

2 2 5 8

PID Burst Deadline Period

1 1 3 6

2 2 5 8

scheduling occurs for 20ms

0ms : P1

1ms : P2

3ms : idle

6ms : P1

7ms : idle

8ms : P2

10ms : idle

12ms : P1

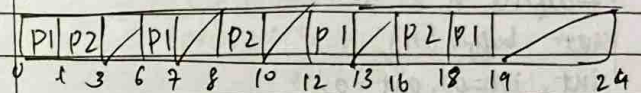
13ms : idle

16ms : P2

18ms : P1

19ms : idle

gantt chart :



8) Producer-consumer

→ #include <stdio.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h>

#define n 8

int buffer[n];

int in=0, out=0;

~~sem_t~~

sem_t empty, full;

pthread_mutex_t s;

int produce_item()

static int item=1;

return item++;

}

void consume_item()

printf("Consumed : %d\n", item);

}

void *producer(void *arg)

int itemp;

while (1) {

itemp = produce_item();

sem_wait(&empty);

pthread_mutex_lock(&s);

buffer[in] = itemp;

printf("Produced : %d at position %d\n",

itemp, in);

in = (in+1)%n;

pthread_mutex_unlock(&s);

sem_post(&full);

sleep(1);

}

void *consume(void *arg) {

int c;

while(1) {

sem_wait(&full);

pthread_mutex_lock(&s);

c = buffer[out];

printf("consumed : %d from position

%d\n", c, out);

out = (out+1)%n;

pthread_mutex_unlock(&s);

sem_post(&empty);

consume_item(c);

sleep(2);

}

}

int main() {

pthread_t prod, cons;

sem_init(&empty, 0, n);

sem_init(&full, 0, 1);

~~pthread_mutex_lock~~

pthread_mutex_init(&s, NULL);

pthread_create(&prod, NULL, producer, NULL);

pthread_create(&cons, NULL, consumer, NULL);

pthread_join(prod, NULL);

pthread_join(cons, NULL);

sem_destroy(&empty);

sem_destroy(&full);

pthread_mutex_destroy(&s);

return 0;

}

output:

produced: 1 at pos 0

consumed: 1 from pos 0

consumed: 1

produced: ~~1~~ at ~~pos 0~~ 2 at pos 1

produced: ~~1~~ from ~~pos 3~~ at pos 2

consumed: 2 from pos 1

consumed: 2

produced: ~~1~~ at ~~pos 4~~ 4 at pos 3

produced: ~~1~~ at ~~pos 5~~ at pos 4

consumed: ~~3~~ from pos 2

consumed: 3

produced: 6 at pos 5

produced: 7 at pos 6

consumed: 4 from pos 3

consumed: ~~3~~ 4

9) Dining Philosopher

⇒ #include <pthread.h>

#include <semaphore.h>

#include <stdio.h>

#include <unistd.h>

#define n 5

sem_t forks[n];

void *philosopher (void * num){

int id = *(int *) num;

while(1){

printf("Philosopher %d is thinking...\n", id);

sleep(1);

if(id == n-1){

sem_wait(&forks[(id+1)%n]);

sem_wait(&forks[id]);

}

else{

sem_wait(&forks[id]);

sem_wait(&forks[(id+1)%n]);

}

printf("Phil %d is eating.\n", id);

sleep(2);

sem_post(&forks[id]);

sem_post(&forks[(id+1)%n]);

printf("Phil %d finished eating and put down forks\n", id);

}

}

```

int main() {
    pthread_t thread_id[n];
    int phil_ids[n];
    for (int i = 0; i < n; i++) {
        sem_init(&forks[i], 0, 1);
        phil_ids[i] = i;
    }
    for (int i = 0; i < n; i++) {
        pthread_create(&thread_id[i], NULL,
                      phil, &phil_ids[i]);
    }
    for (int i = 0; i < n; i++) {
        pthread_join(thread_id[i], NULL);
    }
    for (int i = 0; i < n; i++) {
        sem_destroy(&forks[i]);
    }
    return 0;
}

```

output :

phil 0 is thinking - -
 phil 1 is thinking - -
 phil 2 is thinking - -

phil 0 is eating

phil 0 finished eating and put down forks

~~phil 1 is eating~~

phil 1 finished eating and put down forks

phil 2 is eating

phil 2 is finished eating and put down forks

(10) Bankers Algorithm

```
⇒ #include <stdio.h>
```

```
#define max 10
```

```
int main() {
```

```
    int p, r;
```

```
    int allo[max][max], max[max][max],
```

```
    need[max][max], avail[max];
```

```
    int finish[max] = {0}, safe seq, [max];
```

```
    printf("Enter number of processes");
```

```
    scanf("%d", &p);
```

```
    printf("Enter no of resources");
```

```
    scanf("%d", &r);
```

```
    printf("Enter alloc matrix");
```

```
    for(int i=0; i<p; i++) {
```

```
        for(int j=0; j<r; j++) {
```

```
            scanf("%d", &alloc[i][j]);
```

```
        } printf("Enter max matrix");
```

```
        for(int i=0; i<p; i++) {
```

```
            for(int j=0; j<r; j++) {
```

```
                scanf("%d", &max[i][j]);
```

```
            }
```

```
        } printf("Enter avail matrix");
```

```
        for(int i=0; i<r; i++) {
```

```
            scanf("%d", &avail[i]);
```

```
        }
```

```
    for(int i=0; i<n; i++) {
```

```
        for(int j=0; j<r; j++) {
```

```
            need[i][j] = max[i][j] - alloc[i][j];
```

```
    } int count = 0;
```

```

while (count < p) {
    int found = 0;
    for (int i = 0; i < p; i++) {
        if (!finish[i]) {
            for (int
            for (int j = 0; j < r; j++) {
                if (need[i][j] > available[j])
                    break;
                if (j == r) {
                    for (int k = 0; k < r; k++)
                        available[k] =
                            allocated[i][k];
                    safeseg[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
    }
    if (!found) {
        printf("System is not in safe sequence");
        return 0;
    }
    printf("System is in safe sequence");
    for (int i = 0; i < p; i++)
        printf("P%d ", safeseg[i]);
    return 0;
}

```

Output :

Enter no. of process and resource : 5 5

Enter alloc matrix : 0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter max matrix : 7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter avail resource :

3 3 2

system is in safe sequence

safe sequence : P₁ P₃ P₄ P₀ P₂



(ii) Deadlock detection

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int P, r, i, j;
    printf("Enter no of process and resources\n");
    scanf("%d %d", &P, &r);
    int alloc[P][r], request[P][r], availablr, work[r];
    bool finish[P];
    printf("Enter alloc matrix\n");
    for (i = 0; i < P; i++) {
        for (j = 0; j < r; j++) {
            scanf("%d", &alloc[i][j]);
        }
        printf("Enter request matrix\n");
        for (i = 0; i < P; i++) {
            for (j = 0; j < r; j++) {
                scanf("%d", &request[i][j]);
            }
        }
        printf("Enter avail matrix\n");
        for (i = 0; i < r; i++) {
            scanf("%d", &availablr[i]);
        }
        work[i] = availablr[i];
        for (i = 0; i < P; i++) {
            finish[i] = false;
            for (j = 0; j < r; j++) {
                if (alloc[i][j] != 0) {
                    finish[i] = false;
                }
            }
        }
        break;
    }
}
```

bool found;

do {

found = false;

for (i = 0; i < P; i++) {

if (!finish[i]) {

bool

bool canExecute = true;

for (j = 0; j < r; j++) {

if (request[i][j] > work[j]) {

canExecute = false;

} break;

}

if (canExecute) {

for (j = 0; j < r; j++) {

work[j] += alloc[i][j];

finish[i] = true;

found = true;

}

}

}

} while (found);

bool deadlock = false;

for (i = 0; i < P; i++) {

if (!finish[i]) {

deadlock = true;

printf("process %d is deadlocked\n", i);

}

}

if (!deadlock) printf("no deadlock detected\n");

return 0;

output:

Enter number of processes: 5

Enter no of resources: 3

Enter alloc matrix: 0 1 0

2 0 0

3 0 3

2 1 1

0 0 2

Enter request matrix:

0 0 0

2 0 2

0 0 0

1 0 0

0 0 2

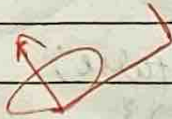
Enter available resources: 0 0 0

~~P1 is deadlocked~~

~~P2 is deadlocked~~

~~P4 is deadlocked~~

No deadlock detected



(12) Proportional scheduling

```
> #include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
typedef struct {
    int id, shares, sum;
} p;
```

```
int main() {
    int n, c, total = 0;
    printf("Enter no of processes: ");
    scanf("%d", &n);
    p p[n];
    for (int i = 0; i < n; i++) {
        printf("Shares for %d: ", i + 1);
        scanf("%d", &p[i].shares);
        p[i].id = i + 1;
        p[i].sum = 0;
        total += p[i].shares;
    }
```

```
printf("Enter no of cycles: ");
scanf("%d", &c);
srand(time(0));
write(c--);
```

```
int win = rand() % total + 1, sum = 0;
for (int i = 0; i < n; i++) {
    if ((sum + p[i].shares) >= win) {
        printf("p. d selected is", p[i].id);
        p[i].sum++;
        break;
    }
```

```
    }
}
```

```
printf("Process schedule:");  
for (i = 0; i < n; i++) {  
    printf("P%d → P%d", PC[i].id,  
           p[i].run);  
}  
}
```

Output:

Enter no of processes: 3

shows for P1: 5

P2: 15

P3: 25

Enter no of cycles: 12

P2

P3

P3

P1

P2

P3

P2

P3

P3

P1

P2

P3

Execution count:

P1 → 2

P2 → 4

P3 → 6

⑬ Contiguous memory allocation techniques

⇒ #include <stdio.h>

```
void firstfit(int *blocks, int m, int *processes,
             int n) {
```

```
    int alloc[50];
```

```
    int temp[50];
```

```
    for (int i=0; i<m; i++) temp[i] = blocks[i];
```

```
    for (int i=0; i<n; i++) alloc[i] = -1;
```

```
    for (int i=0; i<n; i++) {
```

```
        for (int j=0; j<m; j++) {
```

```
            if (temp[j] >= processes[i]) {
```

```
                alloc[i] = j;
```

```
                temp[j] -= processes[i];
```

```
                break;
```

```
            }
```

```
        }
```

```
    }
```

```
    printf("\n First Fit allocation: \n");
```

```
    display(alloc, processes, n);
```

```
}
```

```
void bestfit(int *blocks, int m, int *processes,
            int n) {
```

```
    int alloc[50], temp[50];
```

```
    for (int i=0; i<m; i++) temp[i] = blocks[i];
```

```
    for (int j=0; j<n; j++) alloc[j] = -1;
```

```
    for (int i=0; i<n; i++) { int bestIndex = -1;
```

```
        for (int j=0; j<m; j++) {
```

```
            if (temp[j] >= processes[i]) {
```

```
                if (bestIndex == -1 || temp[j]
```

```
                    < temp[bestIndex])
```

```
                    bestIndex = j;
```

```
            }
```

```
        }
```



```
if (bestIndex != -1) {
    arrc[i] = bestIndex;
    temp[bestIndex] = process[i];
}
```

```
}
printf("\n Best Fit allocation\n");
display (alloc, processes, n);
```

```

void worstFit (int * blocks, int n, int m,
              int * processes) {
    int alloc[50], temp[50];
    for (int i=0; i<m; i++) temp[i] = blocks[i];
    for (int i=0; i<n; i++) alloc[i] = -1;
    for (int i=0; i<n; i++) {
        int worstIndex = -1;
        for (int j=0; j<m; j++) {
            if (temp[j] >= processes[i]) {
                if (worstIndex == -1 || temp[j] > temp[worstIndex])
                    worstIndex = j;
            }
        }
    }
}

```

```
if (wordIndex != -1) {
    alloc[i] = wordIndex;
    temp = wordIndex; // = processing
}
```

```
printf("in worst fit allocation: %d");  
display(alloc, processes, n);
```

```

void display (int* alloc, int* processes, int n) {
    printf("m processes No. It process size It Block\n\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t%d\t\t", i, processes[i], alloc[i]);
        if (alloc[i] != -1)
            printf("%d\n", alloc[i]);
        else
            printf("Not allocated\n");
    }
}

```

```

} void
// main() {

```

```
int m, n, blocks; or data processes;
printf("Enter no of blocks, processes and
size of processes");
scanf("%d, %d, %d", &m, &n, &blocks);
for(int i=0; i<m; i++) scanf("%d", &processes[i]);
printf("Enter no of processes and
size of processes");
scanf("%d", &n);
```

printf("In Memory allocation techniques In
1. First fit In 2. Best fit In 3. Worst
fit ");

```
Scanf("%d", &ch);
```

switser (m) ?

case2: first fit(blocks, m, processes, n);

break:

case 2: $\text{Bertit}(\text{blocks}, m, \text{processes}, n)$

break!

case 3: worst fit (blocks, m, n , processes)

bread;

```
default : printf("Invalid choice!!\n");
```


Output:

Enter no of memory blocks: 5

Enter sizes of memory blocks:

100 500 200 300 600

Enter no of processes: 4

Enter sizes of processes:

212 417 112 426

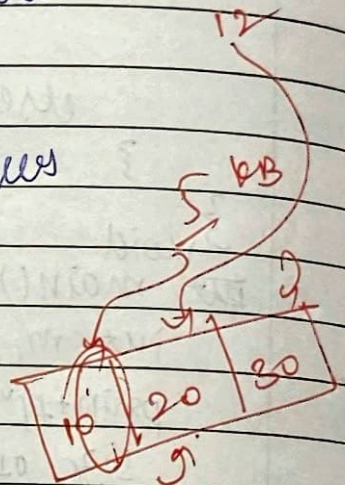
Memory allocation techniques

1. First Fit

2. Best Fit

3. Worst Fit

4.



First Fit Allocation:

Process No.	Process size	Block No
1	212	2
2	417	5
3	112	2
4	426	Not allocated

Best Fit allocation:

Process No.	Process size	Block No.
1	212	4
2	417	2
3	112	3
4	426	5

Worst Fit allocation:

Process No	Process size	Block No.
1	212	5
2	417	2
3	112	5
4	426	Not allocated

④ Page replacement algorithm

⇒ #include <stdio.h>

```
void FIFO (int *pages, int n, int frames) {
    int frame[frames], fault = 0, index = 0,
    found;
    for (int i = 0; i < frames; i++) frame[i] = -1;
    for (int i = 0; i < n; i++) {
        found = 0;
        for (int j = 0; j < frames; j++) {
            if (frame[j] == pages[i]) {
                found = 1; break;
            }
        }
        if (!found) {
            frame[index] = pages[i];
            index = (index + 1) % frames;
            fault++;
        }
    }
}
```

~~printf("\n FIFO page Faults = %d\n", fault);~~

```
void LRU (int *pages, int n, int frames) {
    int frame[frames], time[frames], fault = 0, count = 0;
    for (int i = 0; i < frames; i++) {
        frame[i] = -1;
        time[i] = 0;
    }
    for (int i = 0; i < n; i++) {
        int found = 0;
        for (int j = 0; j < frames; j++) {
```



```
if (found == pages[i]) {
```

```
    count++;
```

```
    time[i] = count;
```

```
    found = 1;
```

```
    break;
```

```
}
```

```
}
```

```
if (!found) {
```

```
    int pos = 0;
```

```
    for (int j = 1; j < frames; j++) {
```

```
        if (time[j] < time[pos])
```

```
            pos = j;
```

```
    }
```

```
    frame[pos] = pages[i];
```

```
    count++;
```

```
    time[pos] = count;
```

```
    fault++;
```

```
}
```

```
}
```

```
printf("\n LRU Page Faults = %d\n",  
    fault);
```

```
}
```

```
void optimal (int * pages, int n, int t) {
```

```
    int fr fr = t;
```

```
    int fault = 0;
```

```
    for (int i = 0; i < t; i++) {
```

```
        fr[i] = -1;
```

```
    } for (int i = 0; i < n; i++) {
```

```
        int found = 0;
```

```
        for (int j = 0; j < t; j++) {
```

```
            if (fr[j] == pages[i]) {
```

```
                found = 1;
```

```
                break;
```

```
        }
```

```
    }
```

```
if (!found) {
```

```
    int pos = -1; farthest = i+1;
```

```
    for (int j = 0; j < t; j++) {
```

```
        int k;
```

```
        for (k = i+1; k < n; k++) {
```

```
            if (fr[k] == pages[i]) break;
```

```
        } if (k == n) {
```

```
            pos = j;
```

```
            break;
```

```
        }
```

```
    } if (k > farthest) {
```

```
        farthest = k;
```

```
        pos = j;
```

```
    }
```

```
}
```

```
if (pos == -1) pos = 0;
```

```
fr[pos] = pages[i];
```

```
fault++;
```

```
}
```

```
}
```

```
printf("\n optimal Page faults = %d\n", fault);
```

```
}
```

```
int main() {
```

```
    int n, frames;
```

```
    printf("Enter no of pages:");
```

```
    scanf("%d", &n);
```

```
    int pages[n];
```

```
    printf("Enter page ref string: \n");
```

```
    for (int i = 0; i < n; i++)
```

```
        scanf("%d", &pages[i]);
```

```
    printf("Enter no of frames:");
```

```
    scanf("%d", &frames);
```



```

FIFO (pages, n, framesframes);
LRU (pages, n, frames);
OPTIMAL (pages, n, frames);
return 0;
}

```

output:

Enter no of pages: 13

Enter page reference string:

7 0 1 2 0 3 0 4 2 3 0 3 2

Enter no of frames

FIFO page faults = 7

LRU page faults = 6

optimal page faults = 6

FIFO:

0	3		
2	2	2	
1	1	1	
0	0	0	
7	3	3	

H

F

-

-

optimal:

3	
2	2
1	1
0	0
7	3

H

F

As 7 never repeats again.

LRU:

0	3	4	
2	2	2	2
1	1	1	4
0	0	0	0
7	3	3	3

H

F

F

as 1 is least recently used.